AI Application Lecture 17

3D Rendering with Hunyuan3D

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

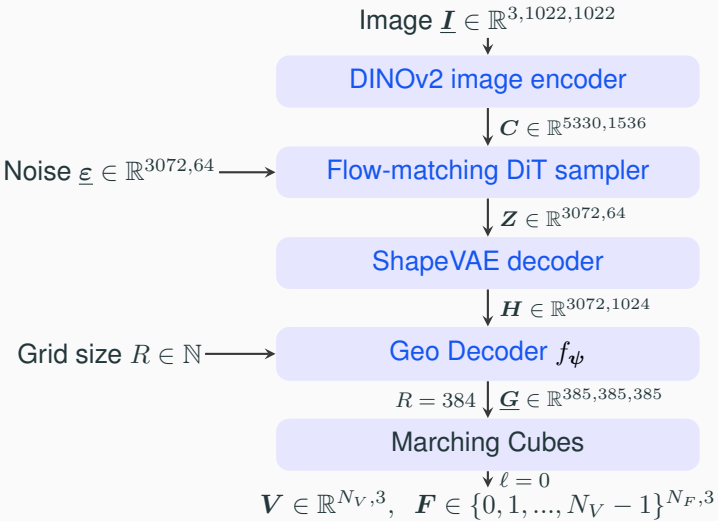
3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

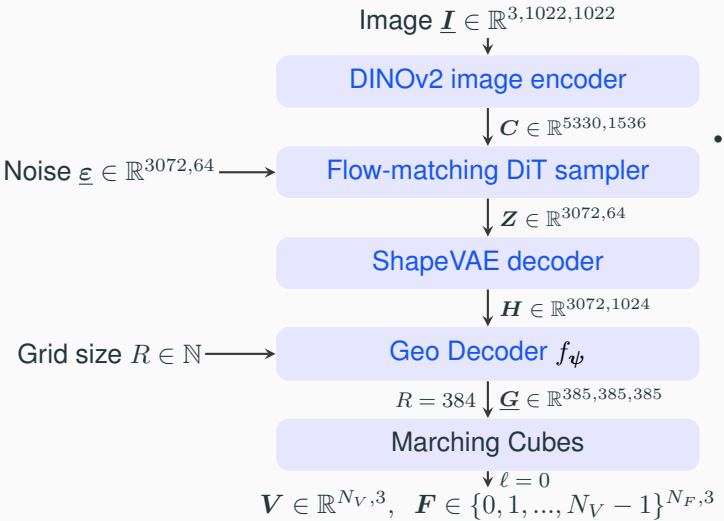
5. How do we train the decoder part?

6. Summary

Hunyuan3D 2.0's pipeline

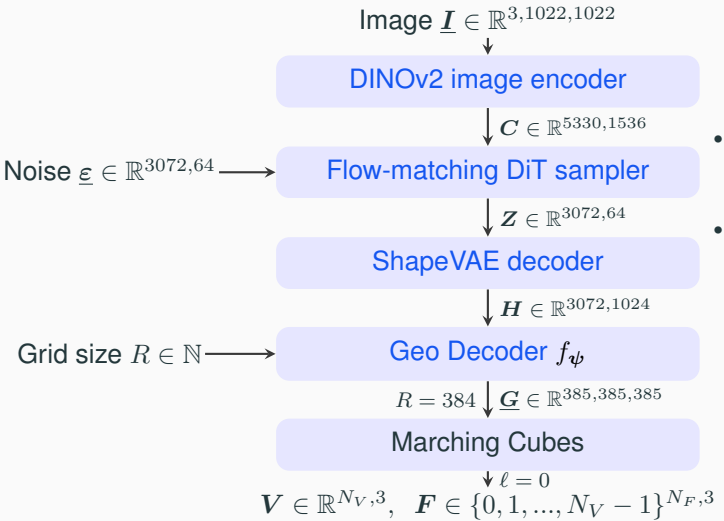


Hunyuan3D 2.0's pipeline



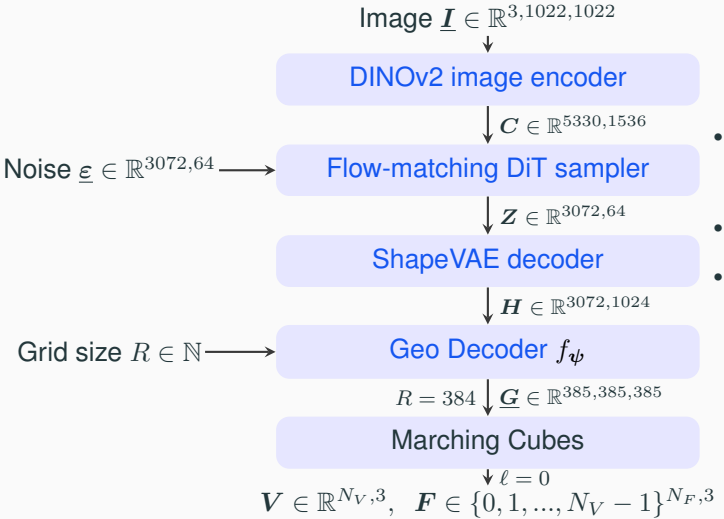
- The image is converted into a tensor $\boldsymbol{H}$.

Hunyuan3D 2.0's pipeline



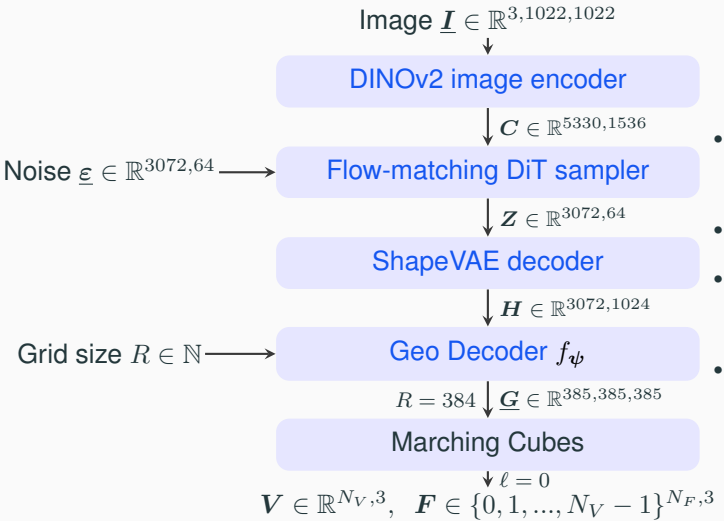
- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\psi}(\cdot, \boldsymbol{H})$ defines a field.

Hunyuan3D 2.0's pipeline



- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\boldsymbol{\psi}}(\cdot, \boldsymbol{H})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.

Hunyuan3D 2.0's pipeline



- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\psi}(\cdot, \boldsymbol{H})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.
- mesh are generated from the field value tensor $\underline{\boldsymbol{G}}$.

Learning outcomes

At the end of the lecture, you will be able to:

Learning outcomes

At the end of the lecture, you will be able to:

- Explain each component's **role** in 3D rendering AI.

Learning outcomes

At the end of the lecture, you will be able to:

- Explain each component's **role** in 3D rendering AI.
- Construct the **specific architecture** of 3D rendering AI.

Learning outcomes

At the end of the lecture, you will be able to:

- Explain each component's **role** in 3D rendering AI.
- Construct the **specific architecture** of 3D rendering AI.
- Construct a **training scheme** of 3D rendering AI.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $\{1, 2, ..., k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_0, a_1, \ldots, a_{n-1})$.

- A function $\boldsymbol{a} : \{0, 1, ..., n-1\} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

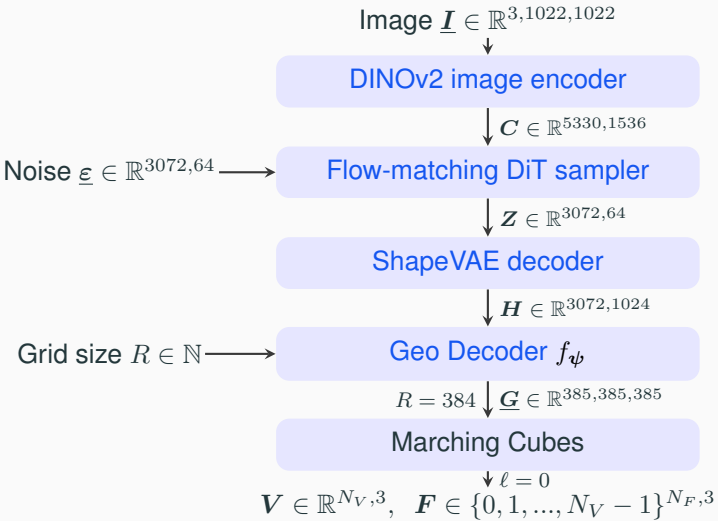
4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

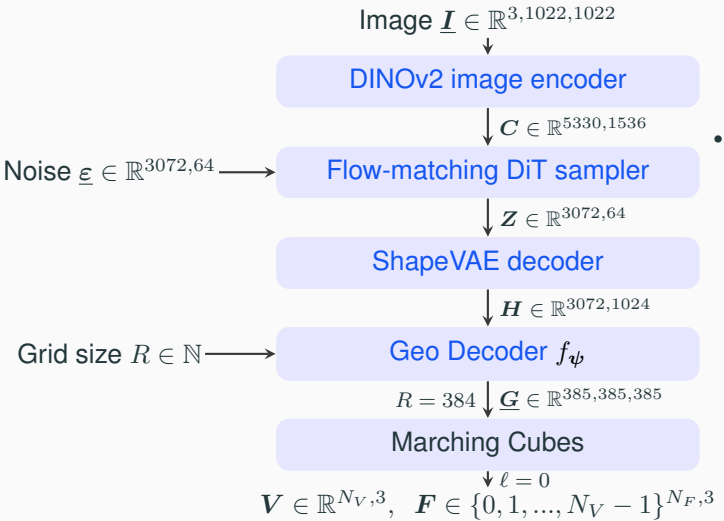
6. Summary

Question: We want to construct a 3D field from an image. What components does a 3D rendering AI consist of?

Hunyuan3D 2.0's pipeline

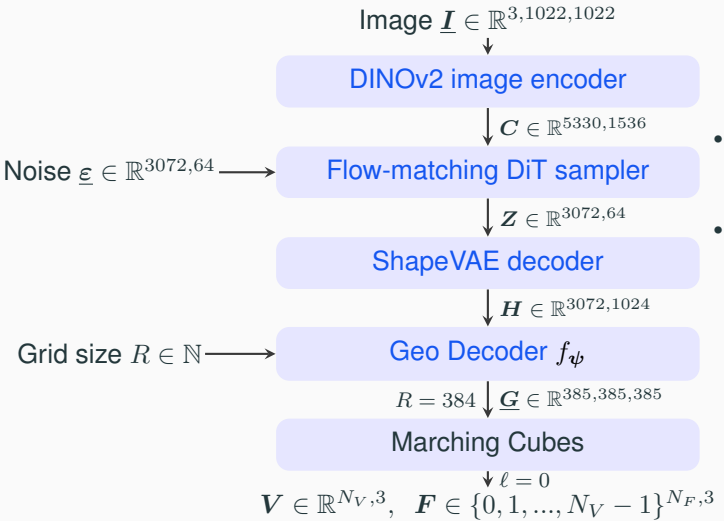


Hunyuan3D 2.0's pipeline



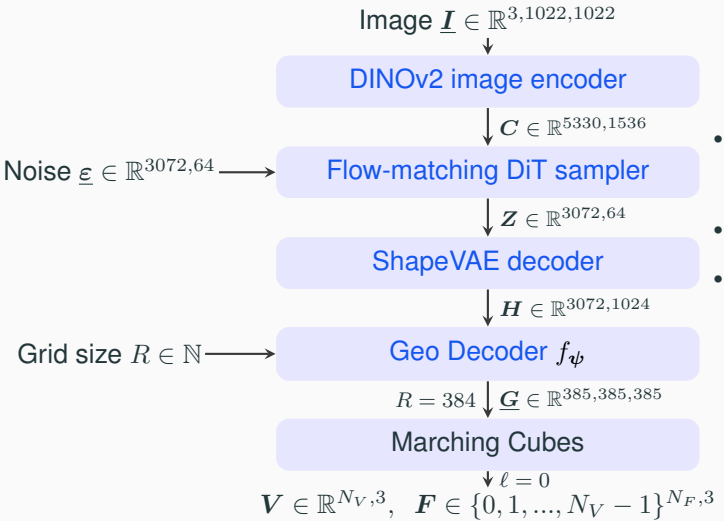
- The image is converted into a tensor $\boldsymbol{H}$.

Hunyuan3D 2.0's pipeline



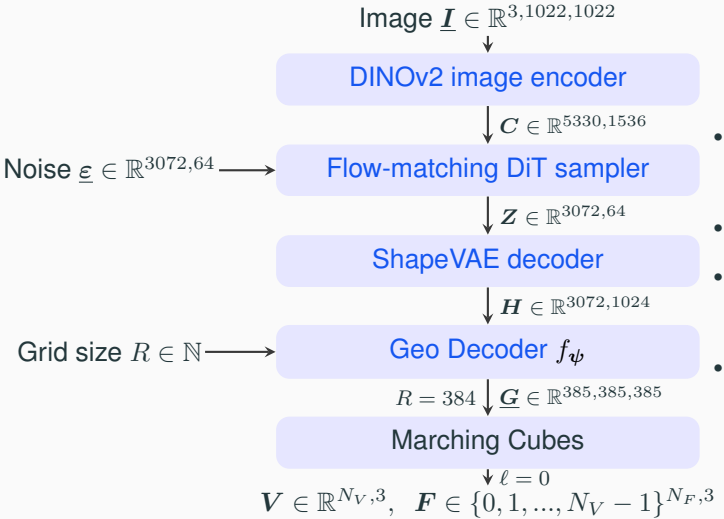
- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\boldsymbol{\psi}}(\cdot, \boldsymbol{H})$ defines a field.

Hunyuan3D 2.0's pipeline



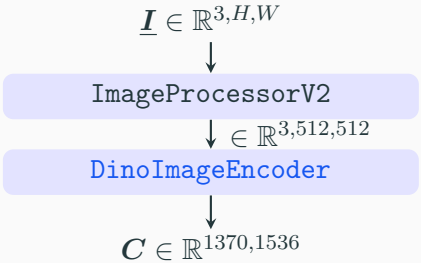
- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\psi}(\cdot, \boldsymbol{H})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.

Hunyuan3D 2.0's pipeline



- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\psi}(\cdot, \boldsymbol{H})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.
- mesh are generated from the field value tensor $\underline{\boldsymbol{G}}$.

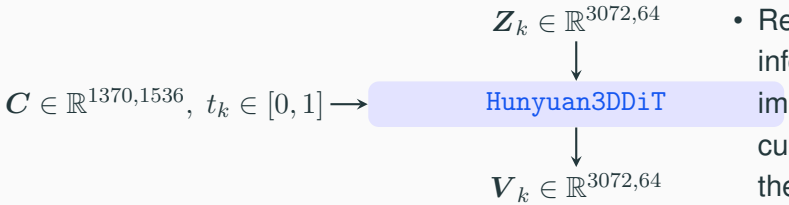
Image encoder: role and interface



- Input image information is converted to 1370 tokens.
- The tokens containing the image information are provided for the DiT in the flow matching generation.

Source: `ImageEncoder.forward`.

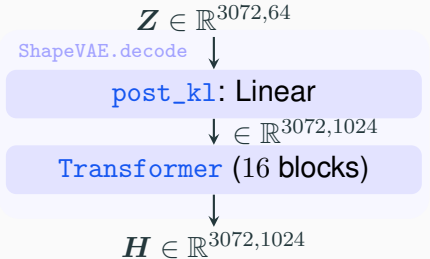
Flow-matching DiT sampler: role



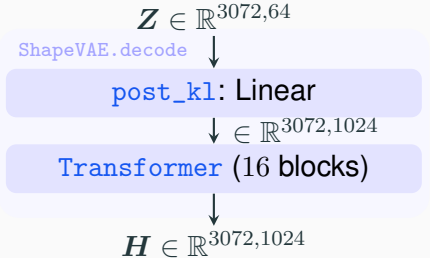
- Receives the image information $\boldsymbol{C}$ from the image encoder and the current time t_k at step k in the flow matching generation.
- Returns the velocity $\boldsymbol{V}_k$ in flow matching generation.
- Flow matching sampling returns the final latent tensor $\boldsymbol{Z}_K$.

Source: `Hunyuan3DDiT.forward`.

ShapeVAE decoder: role

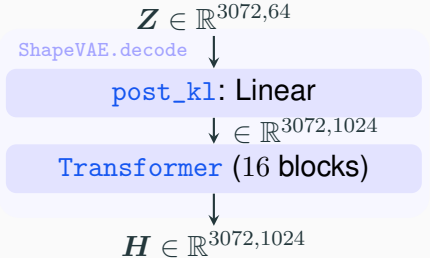


ShapeVAE decoder: role



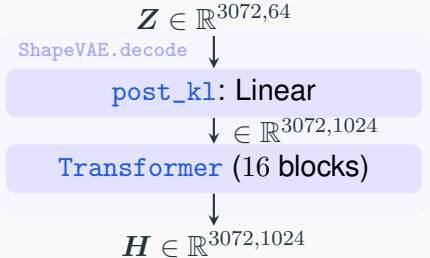
- Each token from the flow matching generator is transformed from 64 to 1024 channels.

ShapeVAE decoder: role



- Each token from the flow matching generator is transformed from 64 to 1024 channels.
- This VAE decoder allows the flow matching generation to work in a low-dimensional space.

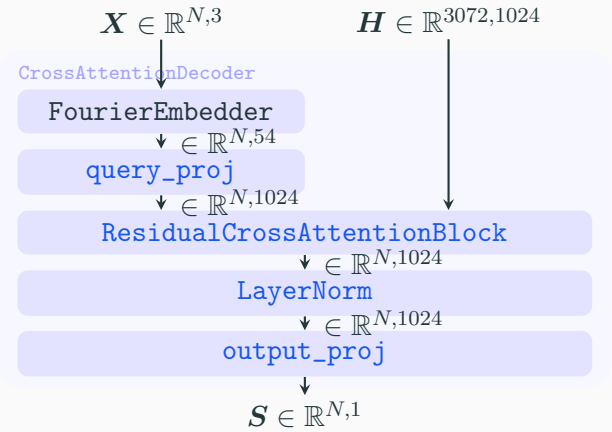
ShapeVAE decoder: role



- Each token from the flow matching generator is transformed from 64 to 1024 channels.
- This VAE decoder allows the flow matching generation to work in a low-dimensional space.
- This computation can be done without a coordinate query.

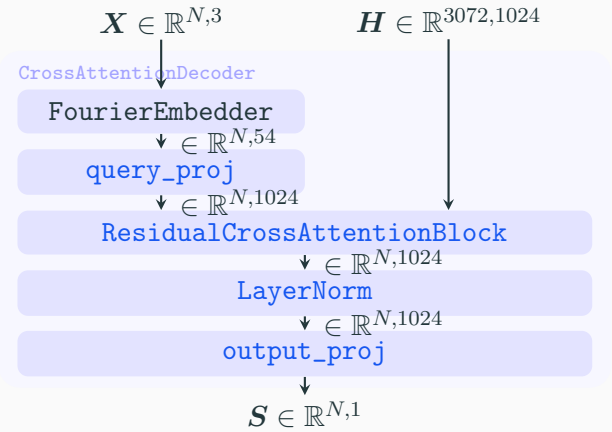
Source: `ShapeVAE.forward`.

Geo decoder: role



- With the image latent information as a condition, the geo decoder works as a 3D field.

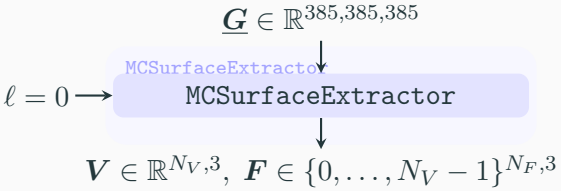
Geo decoder: role



Source: `CrossAttentionDecoder`.

- With the image latent information as a condition, the geo decoder works as a 3D field.
- Replacing the grid changes N, not the already computed $\boldsymbol{H}$.

Marching Cubes: role



- $R = 384$ gives 385 grid samples per axis.
- Extract the zero isosurface by interpolating edge crossings.
- No learnable parameters are required.
- Output vertex and triangle counts vary.

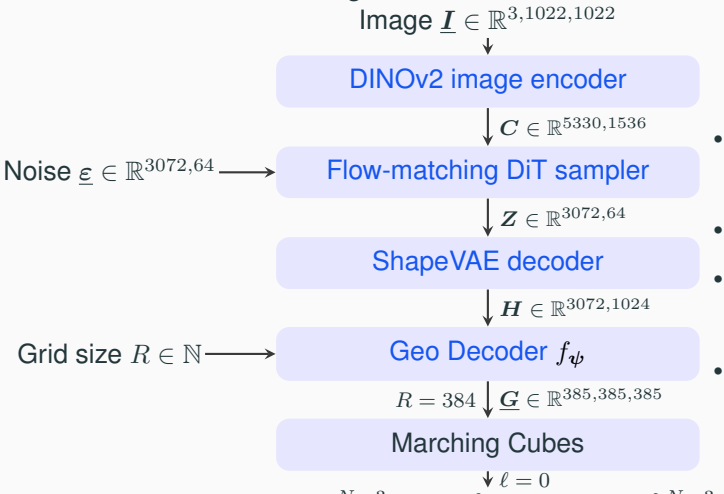
Source: `MCSurfaceExtractor.run`.

Components of 3D rendering AI — answer

What does a 3D rendering AI consist of? **Answer**:

Components of 3D rendering AI — answer

What does a 3D rendering AI consist of? **Answer**:



- The image is converted into a tensor $\boldsymbol{H}$.
- $f_{\boldsymbol{\psi}}(\cdot, \boldsymbol{H})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.
- mesh are generated from the field value tensor $\underline{\boldsymbol{G}}$.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

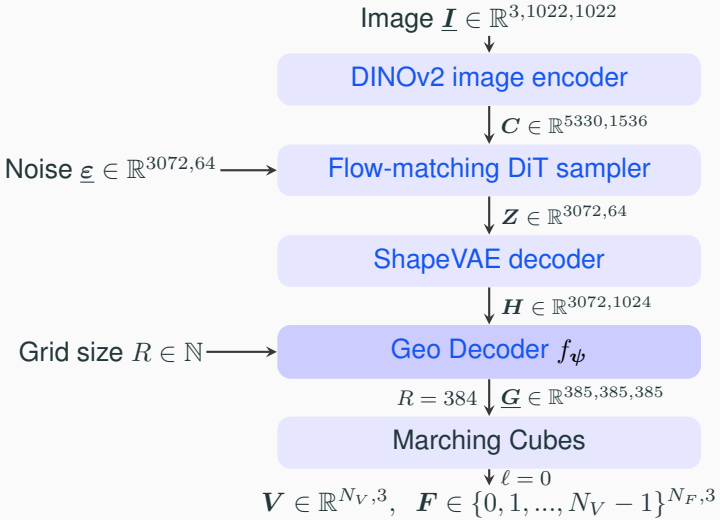
4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

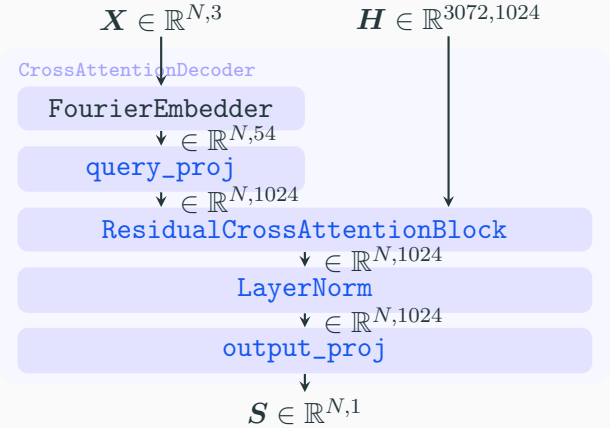
6. Summary

Question: What architecture does each component adopt to achieve the functionality? Specifically, how does it construct a 3D field?

Hunyuan3D 2.0's pipeline



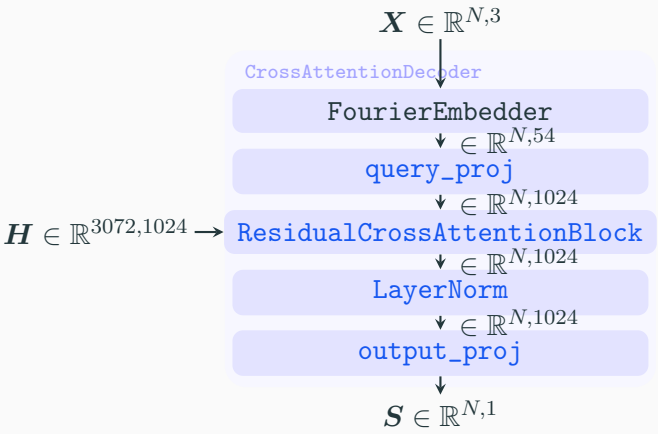
Geo decoder (CrossAttentionDecoder)



Source: `CrossAttentionDecoder`.

- The query sequence length N is independent of 3072.
- Replacing the grid changes N, not the already computed $\boldsymbol{H}$.

CrossAttentionDecoder: query-dependent stages



Source: `CrossAttentionDecoder.forward`.

- The coordinate count N is arbitrary and may be chunked.
- The latent context has shape $\boldsymbol{H} \in \mathbb{R}^{3072,1024}$.
- The output projection is $\boldsymbol{W}_o \in \mathbb{R}^{1,1024}$.
- This is the coordinate-dependent portion of the full VAE decoder.

FourierEmbedder: query coordinates

For a position $\boldsymbol{x} = (x_1, x_2, x_3)^\top \in \mathbb{R}^3$, the released checkpoint uses eight frequencies and includes the raw coordinate:

$$f_j = 2^j, \quad j = 0, \ldots, 7,$$
$$\boldsymbol{\phi}(\boldsymbol{x}) = \mathrm{Concat}\left(\boldsymbol{x},\ (\sin(f_j x_a))_{a,j},\ (\cos(f_j x_a))_{a,j}\right) \in \mathbb{R}^{51}.$$

For N coordinate rows, $\boldsymbol{\Phi}(X) \in \mathbb{R}^{N,51}$. The query vectors are

$$\boldsymbol{q}_i^{(0)} = \boldsymbol{W}_{\mathrm{query}} \boldsymbol{\phi}(\boldsymbol{x}_i) + \boldsymbol{b}_{\mathrm{query}} \in \mathbb{R}^{1024},$$

$\boldsymbol{W}_{\mathrm{query}} \in \mathbb{R}^{1024,51}$. This deterministic Fourier transform has no trainable parameters.

Source: `FourierEmbedder`.

Exercise — Fourier Embedder

Exercise

For this exercise, use the three frequencies $f_j = 2^j$ $(j = 0, 1, 2)$. For $\boldsymbol{x} = (x_1, x_2, x_3)^\top \in \mathbb{R}^3$, define

$$\boldsymbol{\phi}(\boldsymbol{x}) = \mathrm{Concat}\big(\boldsymbol{x}, (\sin(f_j x_a))_{a=1,2,3;j=0,1,2}, (\cos(f_j x_a))_{a=1,2,3;j=0,1,2}\big) \in \mathbb{R}^{21}.$$

Compute the Fourier embeddings of the two coordinates

$$\boldsymbol{x}^{(0)} = (\pi/6, 0, 0)^\top, \qquad \boldsymbol{x}^{(1)} = (\pi/2, -\pi/6, \pi/4)^\top.$$

For each coordinate $a = 1, 2, 3$, write its three sine entries $(\sin(2^j x_a))_{j=0}^2$ and three cosine entries $(\cos(2^j x_a))_{j=0}^2$.

Source: `FourierEmbedder.forward`; `include_pi: false`.

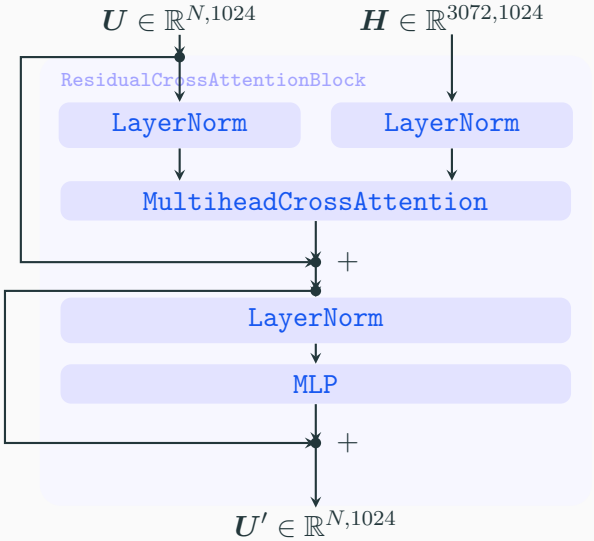
Exercise — Fourier Embedder

Answer: Write $\boldsymbol{\phi}(\boldsymbol{x}) = (\boldsymbol{x}, \boldsymbol{s}(\boldsymbol{x}), \boldsymbol{c}(\boldsymbol{x}))$, where $\boldsymbol{s}, \boldsymbol{c} \in \mathbb{R}^9$. For each row, the entries are ordered by $j = 0, 1, 2$.

	a	$(\sin(2^j x_a))_{j=0}^2$	$(\cos(2^j x_a))_{j=0}^2$
$\boldsymbol{x}^{(0)}$	1	$(\frac{1}{2}, \frac{\sqrt{3}}{2}, \frac{\sqrt{3}}{2})$	$(\frac{\sqrt{3}}{2}, \frac{1}{2}, -\frac{1}{2})$
	2	$(0, 0, 0)$	$(1, 1, 1)$
	3	$(0, 0, 0)$	$(1, 1, 1)$
$\boldsymbol{x}^{(1)}$	1	$(1, 0, 0)$	$(0, -1, 1)$
	2	$(-\frac{1}{2}, -\frac{\sqrt{3}}{2}, -\frac{\sqrt{3}}{2})$	$(\frac{\sqrt{3}}{2}, \frac{1}{2}, -\frac{1}{2})$
	3	$(\frac{\sqrt{2}}{2}, 1, 0)$	$(\frac{\sqrt{2}}{2}, 0, -1)$

Thus both embeddings belong to $\mathbb{R}^{21}$, because $3 + 3 \cdot 3 + 3 \cdot 3 = 21$. For the released model, $j = 0, \ldots, 7$ and the corresponding embedding dimension is 51.

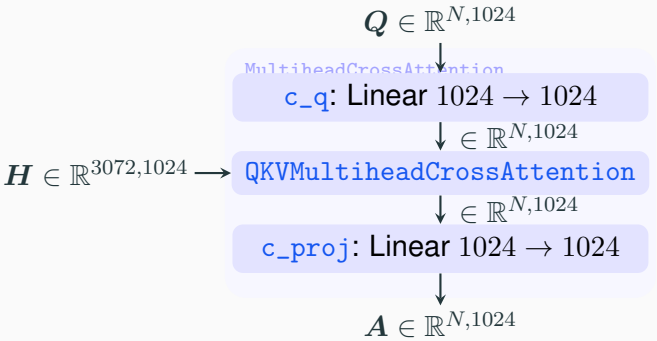
ResidualCrossAttentionBlock: latent lookup



- Query input: $\boldsymbol{U} \in \mathbb{R}^{N,1024}$.
- Context input: $\boldsymbol{H} \in \mathbb{R}^{3072,1024}$.
- Query and context have independent `LayerNorm` modules (`ln_1`, `ln_2`).
- Cross-attention uses N queries and 3072 keys and values.
- The final `MLP` follows `LayerNorm` (`ln_3`).

Source: `ResidualCrossAttentionBlock.forward`.

MultiheadCrossAttention: separate Q and KV



- The context receives `c_kv`: $1024 \to 2048$ per latent token.
- Query vectors come from 3D positions; key/value vectors from shape latents.
- Each of 16 heads uses 64 dimensions.
- Attention score size per head is $N \times 3072$.

Source: `MultiheadCrossAttention`.

QKVMultiheadCrossAttention: point-wise values

For query point $i \in \{1, \dots, N\}$ and head $h \in \{1, \dots, 16\}$,

$$\boldsymbol{q}_{i,h} = \mathrm{LN}_{Q,h}(\boldsymbol{W}_h^Q \boldsymbol{\phi}_i + \boldsymbol{b}_h^Q) \in \mathbb{R}^{64},$$

$$\boldsymbol{k}_j^{[h]} = \mathrm{LN}_{K,h}(\boldsymbol{W}_h^K \boldsymbol{h}_j + \boldsymbol{b}_h^K), \quad \boldsymbol{v}_j^{[h]} = \boldsymbol{W}_h^V \boldsymbol{h}_j + \boldsymbol{b}_h^V \in \mathbb{R}^{64}.$$

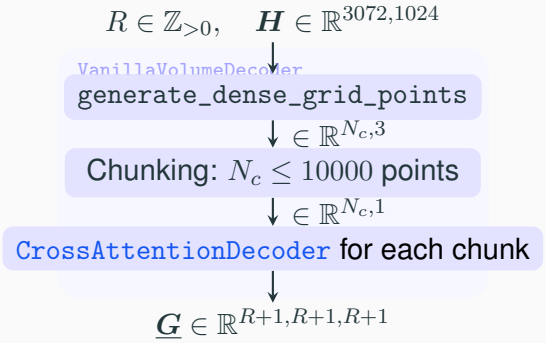
Here $\boldsymbol{\phi}_i, \boldsymbol{h}_j \in \mathbb{R}^{1024}$ and all projection matrices have size 64×1024.

$$a_{ij}^{(h)} = \frac{\exp\left(\boldsymbol{q}_{i,h}^\top \boldsymbol{k}_j^{[h]} / 8\right)}{\sum_{r=1}^{3072} \exp\left(\boldsymbol{q}_{i,h}^\top \boldsymbol{k}_r^{[h]} / 8\right)}, \quad \boldsymbol{y}_{i,h} = \sum_{j=1}^{3072} a_{ij}^{(h)} \boldsymbol{v}_j^{[h]}.$$

Thus $\boldsymbol{y}_i = \mathrm{Concat}_h \, \boldsymbol{y}_{i,h} \in \mathbb{R}^{1024}$ is defined for every requested coordinate independently.

Source: `QKVMultiheadCrossAttention`.

VanillaVolumeDecoder: dense evaluation



- Grid coordinates lie in the bounding box.
- For $R = 384$, the output is $385 \times 385 \times 385$.
- Changing R changes queried positions.
- All chunks reuse $\boldsymbol{H}$.

Source: `VanillaVolumeDecoder`.

Constructing a 3D field — Answer

Question: Specifically, how does a 3D rendering AI construct a 3D field?

Answer: Hunyuan3D geodecoder applies Fourier embedding, and get the field value through the cross attention between the embedding and the information from the latent representation of the input image.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Question: We want to construct a decoder that outputs a field value. However,

Question: We want to construct a decoder that outputs a field value. However, the training dataset is expected to consist of the shape data, not field values.

How

Question: We want to construct a decoder that outputs a field value. However, the training dataset is expected to consist of the shape data, not field values.

How can we train a 3D field NN from the shape data?

Learning strategy of VAE part of Hunyuan3D

In an ordinary VAE for a raw data space $\mathcal{Y}$ and a latent space $\mathcal{Z}$,

- Encoder : $\mathcal{Y} \to \mathcal{Z}$, and
- Decoder : $\mathcal{Z} \to \mathcal{Y}$.

They are an “inverse” of each other.

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

- Pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector.

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

- Pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector.
- 3D field $\boldsymbol{S}$.

Then, we let

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

- Pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector.
- 3D field $\boldsymbol{S}$.

Then, we let

- The encoder receive pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector and return a latent representation $\boldsymbol{Z}$.

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

- Pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector.
- 3D field $\boldsymbol{S}$.

Then, we let

- The encoder receive pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector and return a latent representation $\boldsymbol{Z}$.
- The decoder receive a latent representation $\tilde{\boldsymbol{Z}}$ and return a 3D field $\widehat{\boldsymbol{S}}$.

Learning strategy of VAE part of Hunyuan3D

Hunyuan3D uses a non-inverse strategy.

From real shape data, we create:

- Pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector.
- 3D field $\boldsymbol{S}$.

Then, we let

- The encoder receive pairs $(\boldsymbol{p}_i, \boldsymbol{n}_i)_{i=0}^{N_P-1}$ of a surface point and the normal vector and return a latent representation $\boldsymbol{Z}$.
- The decoder receive a latent representation $\tilde{\boldsymbol{Z}}$ and return a 3D field $\widehat{\boldsymbol{S}}$.

The loss function is about the difference between $\boldsymbol{S}$ and $\widehat{\boldsymbol{S}}$.

Mesh samples: positions, normals, and queries

For a nondegenerate face (a_j, b_j, c_j), define its unit normal

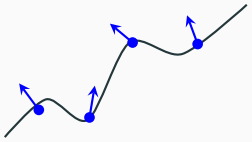
$$\boldsymbol{n}_j = \frac{(\boldsymbol{v}_{b_j} - \boldsymbol{v}_{a_j}) \times (\boldsymbol{v}_{c_j} - \boldsymbol{v}_{a_j})}{\|(\boldsymbol{v}_{b_j} - \boldsymbol{v}_{a_j}) \times (\boldsymbol{v}_{c_j} - \boldsymbol{v}_{a_j})\|_2} \in \mathbb{R}^3.$$

Sample positions and normals on the mesh surface:

$$\boldsymbol{P}, \boldsymbol{N} \in \mathbb{R}^{N_P, 3}.$$

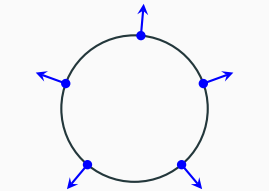
Rows of $\boldsymbol{P}$ and $\boldsymbol{N}$ describe a sampled position and its corresponding surface normal.

Source: Hunyuan3D 2.0, Sec. 3.1.

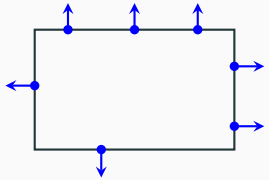


Surface samples $\boldsymbol{p}_i$
Normals $\boldsymbol{n}_i$

Surface samples and normals: two-dimensional examples



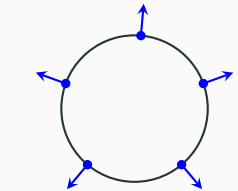
Circle



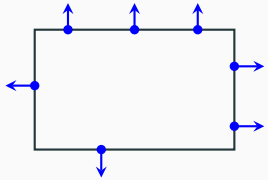
Rectangle

(Two dimensional example for simplicity)

Surface samples and normals: two-dimensional examples



Circle



Rectangle

(Two dimensional example for simplicity)

- Blue dots: $\boldsymbol{p}_i \in \mathbb{R}^2$ on the boundary;
- Arrows: outward unit normals $\boldsymbol{n}_i \in \mathbb{R}^2$.

Source: Hunyuan3D 2.0, Sec. 3.1.

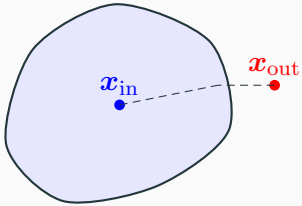
Mesh data to field: signed distance

For a closed, consistently oriented mesh, let Ω_M be its bounded interior and $\mathcal{S}_M$ its surface. Define the distance

$$d_M(\boldsymbol{x}) = \min_{\boldsymbol{y} \in \mathcal{S}_M} \|\boldsymbol{x} - \boldsymbol{y}\|_2.$$

The supervised scalar field is

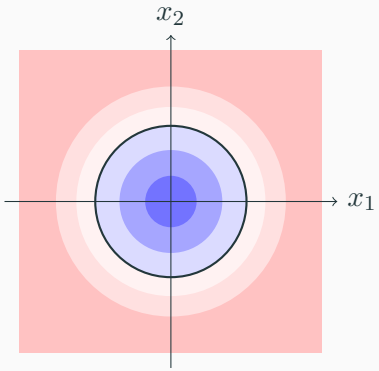
$$s_M(\boldsymbol{x}) = \begin{cases} -d_M(\boldsymbol{x}), & \boldsymbol{x} \in \Omega_M, \\ d_M(\boldsymbol{x}), & \boldsymbol{x} \notin \Omega_M. \end{cases}$$



$s_M(\boldsymbol{x}_{\text{in}}) < 0$
$s_M(\boldsymbol{x}_{\text{out}}) > 0$

Source: Hunyuan3D 2.0, Sec. 3.1.

Signed distance field example: a circle



For a circle of radius $r > 0$ centered at the origin, the signed distance is

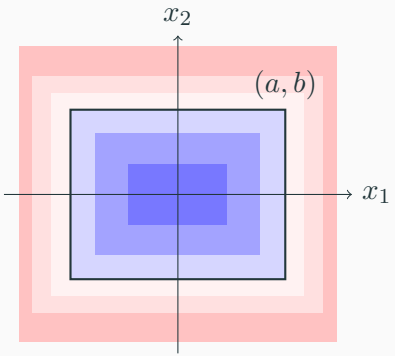
$$s_{\text{circle}}(\boldsymbol{x}) = \|\boldsymbol{x}\|_2 - r, \quad \boldsymbol{x} \in \mathbb{R}^2.$$

Blue: $s < 0$ (inside).

Red: $s > 0$ (outside).
The black contour has $s = 0$.

Source: Hunyuan3D 2.0, Sec. 3.1; the signed distance follows the Euclidean-distance definition.

Signed distance field example: a rectangle



For the rectangle $[-a, a] \times [-b, b]$ with $a, b > 0$, define

$$\boldsymbol{d} = \begin{pmatrix} |x_1| - a \\ |x_2| - b \end{pmatrix} \in \mathbb{R}^2.$$

Writing $\boldsymbol{d}_+ = \max(\boldsymbol{d}, \boldsymbol{0})$ componentwise and $m = \max(d_1, d_2)$, its signed distance is

$$s_{\text{box}}(\boldsymbol{x}) = \|\boldsymbol{d}_+\|_2 + \min(m, 0).$$

Blue: $s < 0$; red: $s > 0$. The black boundary has $s = 0$.

Source: Hunyuan3D 2.0, Sec. 3.1; the signed distance follows the Euclidean-distance definition.

Mesh samples: field queries

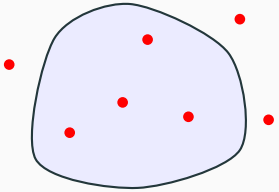
Sample field evaluation positions

$$\boldsymbol{X} = (\boldsymbol{x}_j^\top)_{j=1}^K \in \mathbb{R}^{K,3}$$

in the volume and near the mesh surface.
Their labels are

$$\boldsymbol{s}_M = (s_M(\boldsymbol{x}_1), \ldots, s_M(\boldsymbol{x}_K))^\top \in \mathbb{R}^K.$$

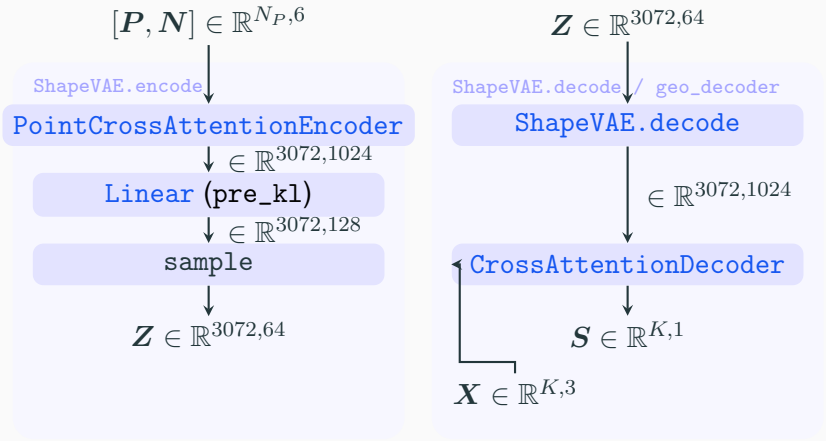
The encoder takes $(\boldsymbol{P}, \boldsymbol{N})$ as input; field reconstruction uses $(\boldsymbol{X}, \boldsymbol{s}_M)$ as supervision.



$\boldsymbol{x}_j \in \mathbb{R}^3$
$s_M(\boldsymbol{x}_j) \in \mathbb{R}$

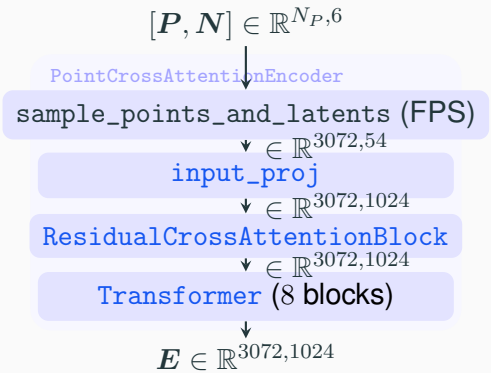
Source: Hunyuan3D 2.0, Sec. 3.1.

Learning ShapeVAE: decoder and encoder together



Source: `ShapeVAE.encode, decode.`

PointCrossAttentionEncoder: latent inference



Source: `PointCrossAttentionEncoder.forward`.

- Input: sampled positions and normals.
- FPS: 3072 queries from two subsets.
- Cross-attention reads surface features.
- Transformer mixes latent tokens.

Latent reparameterization

The encoder output $\boldsymbol{E} \in \mathbb{R}^{3072,1024}$ is projected by $\boldsymbol{W}_{\text{pre}} \in \mathbb{R}^{128,1024}$ to $\boldsymbol{\mu}, \boldsymbol{\ell} \in \mathbb{R}^{3072,64}$, with $\boldsymbol{\ell}$ representing log-variance entries. For a noise array $\boldsymbol{\varepsilon} \in \mathbb{R}^{3072,64}$ with independent entries of mean 0 and variance 1, define

$$\widetilde{\boldsymbol{Z}} = \boldsymbol{\mu} + \exp(\boldsymbol{\ell}/2) \odot \boldsymbol{\varepsilon} \in \mathbb{R}^{3072,64}.$$

This reparameterization permits differentiation of the reconstruction loss with respect to the encoder parameters at a fixed $\boldsymbol{\varepsilon}$.

Source: `ShapeVAE.encode`.

SDF supervision: regularized reconstruction loss

For training mesh M, let $\boldsymbol{X} = (\boldsymbol{x}_j^\top)_{j=1}^K \in \mathbb{R}^{K,3}$ be the sampled query coordinates, $\boldsymbol{s}_M = (s_M(\boldsymbol{x}_j))_{j=1}^K \in \mathbb{R}^K$. Denote the predicted scalar value from the noisy latent $\widetilde{\boldsymbol{Z}}$ by $\widehat{\boldsymbol{s}}_{M,j}$.

The objective function is:

$$\mathcal{L}(\phi, \psi, \omega) = \mathbb{E}_{M,\boldsymbol{\varepsilon},\boldsymbol{X}} \left[\frac{1}{K} \sum_{j=1}^{K} (\widehat{s}_M(\boldsymbol{x}_j) - s_M(\boldsymbol{x}_j))^2 + \gamma \mathcal{L}_{\text{reg}}(\boldsymbol{\mu}, \boldsymbol{\ell}) \right].$$

Here,

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\mu}, \boldsymbol{\ell}) = \frac{1}{2} \sum_{i=1}^{3072} \sum_{a=1}^{64} \left(\mu_{ia}^2 + e^{\ell_{ia}} - 1 - \ell_{ia} \right).$$

Source: Sec. 3.1, Eq. (1).

Training VAE in 3D rendering AI

Question: How can we train a 3D field NN from shape data?

Answer: From shape data, we sample surface points and the normal vectors, and 3D signed distance field values. During the training,

Training VAE in 3D rendering AI

Question: How can we train a 3D field NN from shape data?

Answer: From shape data, we sample surface points and the normal vectors, and 3D signed distance field values. During the training,

- The pairs of a point and the normal vector are used as input of the VAE encoder.

Training VAE in 3D rendering AI

Question: How can we train a 3D field NN from shape data?

Answer: From shape data, we sample surface points and the normal vectors, and 3D signed distance field values. During the training,

- The pairs of a point and the normal vector are used as input of the VAE encoder.
- The signed distance field values are used as supervision to evaluate the VAE decoder outputs.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What components does a 3D rendering AI consist of?

4. What architecture does each component adopt to achieve the functionality?

5. How do we train the decoder part?

6. Summary

Summary

Summary

- Image encoder provides the DiT with image information. DiT generates a latent representation. The geo decoder receives the representation as a condition and works as a 3D field.

Summary

- Image encoder provides the DiT with image information. DiT generates a latent representation. The geo decoder receives the representation as a condition and works as a 3D field.
- Hunyuan3D geodecoder applies Fourier embedding, and get the field value through the cross attention between the embedding and the information from the latent representation of the input image.

Summary

- Image encoder provides the DiT with image information. DiT generates a latent representation. The geo decoder receives the representation as a condition and works as a 3D field.
- Hunyuan3D geodecoder applies Fourier embedding, and get the field value through the cross attention between the embedding and the information from the latent representation of the input image.
- From shape data, we sample surface points and the normal vectors as inputs for the VAE encoder, and 3D signed distance field values as supervision to evaluate the VAE decoder outputs.

References i

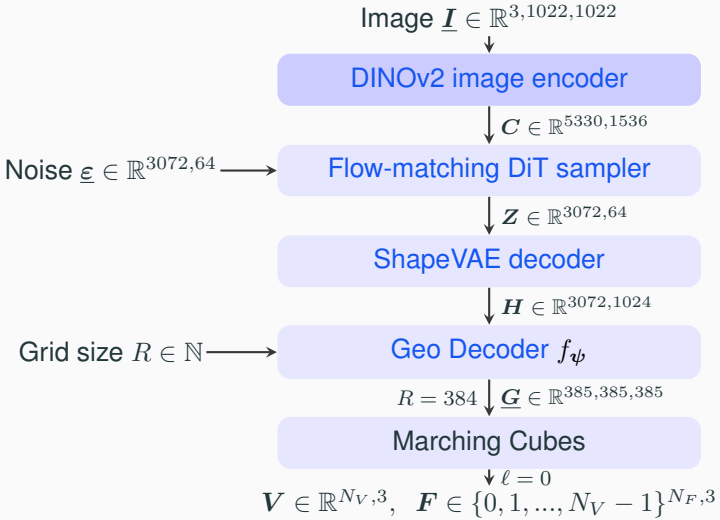
Outline

7. Appendix

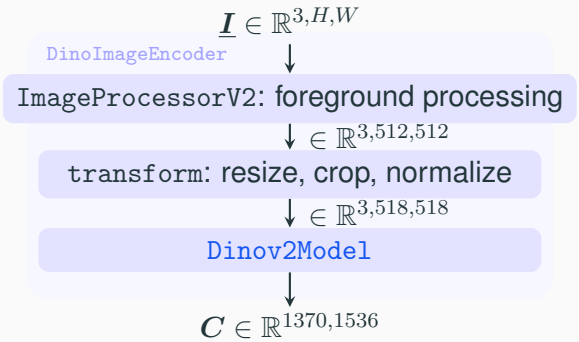
Outline

7. Appendix

Hunyuan3D 2.0's pipeline



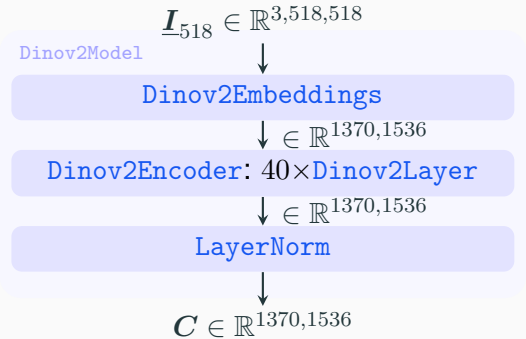
ImageProcessorV2 and DinoImageEncoder



- `ImageProcessorV2` standardizes object scale, location, and background.
- `DinoImageEncoder` inherits `ImageEncoder`; its `Dinov2Model` is frozen.
- $\boldsymbol{C}$ contains one CLS token and 37^2 patch tokens.

Source: `ImageEncoder`, `DinoImageEncoder`.

Dinov2Model: patch and transformer stages



- Patch size and stride are 14×14.
- Embeddings retain spatial order using positional information.
- 40 transformer layers have width 1536 and 24 heads.
- Frozen layers supply conditions for shape generation.

Source: `Dinov2Embeddings`.

Dinov2PatchEmbeddings: 2D convolution

For output channel $o \in \{1, \ldots, 1536\}$ and patch index (i, j), where $i, j = 0, 1, ..., 36$,

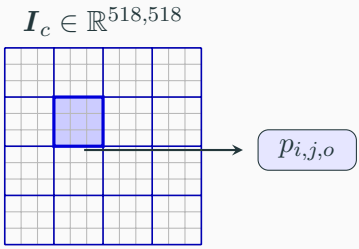
$$p_{i,j,o} = b_o + \sum_{c=1}^{3} \sum_{a,b=0}^{13} K_{o,c,a,b}\, I_{c,14i+a,14j+b}.$$

$$\boldsymbol{P} \in \mathbb{R}^{37^2,1536}, \qquad \underline{\boldsymbol{K}} \in \mathbb{R}^{1536,3,14,14}.$$

A learned CLS vector and positional embedding give

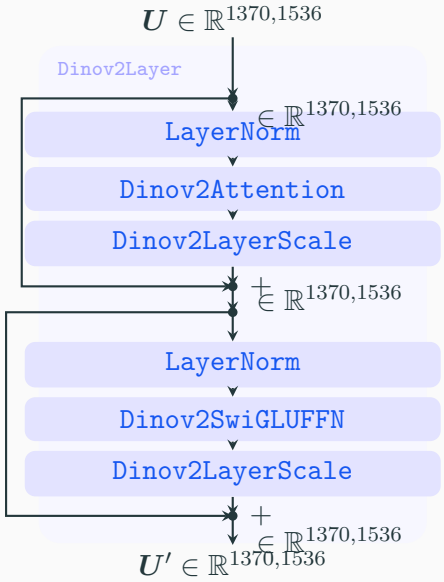
$$\boldsymbol{C}^{(0)} = \mathrm{Concat}(\boldsymbol{c}_{\mathrm{cls}}, \boldsymbol{P}) + \boldsymbol{E}_{\mathrm{pos}} \in \mathbb{R}^{1370,1536}.$$

Source: `Dinov2PatchEmbeddings`.



12×12 pixels: 3×3 patches
Actual: 518×518, 14×14 patch

Dinov2Layer: two residual branches



- Both residual additions require identical tensor shapes.
- `Dinov2LayerScale` multiplies output channels by a learned vector.
- $\boldsymbol{U}_1 = \boldsymbol{U} + \boldsymbol{\lambda}_1 \odot \mathrm{Attn}(\mathrm{LN}(\boldsymbol{U}))$.
- $\boldsymbol{U}' = \boldsymbol{U}_1 + \boldsymbol{\lambda}_2 \odot \mathrm{FFN}(\mathrm{LN}(\boldsymbol{U}_1))$.

Dinov2Attention: token-wise computation

For token $i \in \{1, \dots, 1370\}$, head $h \in \{1, \dots, 24\}$, $d_h = 64$,

$$\boldsymbol{q}_i^{[h]} = \boldsymbol{W}_{\mathsf{Q}}^{[h]} \boldsymbol{u}_i + \boldsymbol{b}_{\mathsf{Q}}^{[h]}, \quad \boldsymbol{k}_i^{[h]} = \boldsymbol{W}_{\mathsf{K}}^{[h]} \boldsymbol{u}_i + \boldsymbol{b}_{\mathsf{K}}^{[h]}, \quad \boldsymbol{v}_i^{[h]} = \boldsymbol{W}_{\mathsf{V}}^{[h]} \boldsymbol{u}_i + \boldsymbol{b}_{\mathsf{V}}^{[h]},$$

$$\boldsymbol{q}_i^{[h]}, \boldsymbol{k}_i^{[h]}, \boldsymbol{v}_i^{[h]} \in \mathbb{R}^{64}, \qquad \boldsymbol{W}_{\mathsf{Q}}^{[h]}, \boldsymbol{W}_{\mathsf{K}}^{[h]}, \boldsymbol{W}_{\mathsf{V}}^{[h]} \in \mathbb{R}^{64,1536}.$$

$$a_{ij}^{[h]} = \frac{\exp\left((\boldsymbol{q}_i^{[h]})^\top \boldsymbol{k}_j^{[h]} / 8 \right)}{\sum_{r=1}^{1370} \exp\left((\boldsymbol{q}_i^{[h]})^\top \boldsymbol{k}_r^{[h]} / 8 \right)}, \quad \boldsymbol{o}_i^{[h]} = \sum_{j=1}^{1370} a_{ij}^{[h]} \boldsymbol{v}_j^{[h]} \in \mathbb{R}^{64}.$$

The 24 head outputs concatenate into $\boldsymbol{o}_i \in \mathbb{R}^{1536}$ and are projected back to 1536 channels.

Source: `Dinov2Attention`.

Dinov2SwiGLUFFN and SiLU

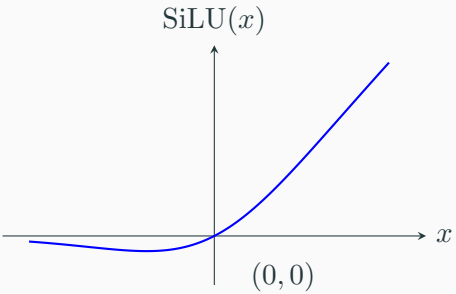
For each $\boldsymbol{u}_i \in \mathbb{R}^{1536}$, the hidden width is

$$m = \left\lceil \frac{1536 \cdot 4 \cdot 2/3}{8} \right\rceil 8 = 4096.$$

$$\begin{aligned} \boldsymbol{a}_i &= \boldsymbol{W}_g \boldsymbol{u}_i + \boldsymbol{b}_g, \\ \boldsymbol{b}_i &= \boldsymbol{W}_u \boldsymbol{u}_i + \boldsymbol{b}_u, \\ \boldsymbol{o}_i &= \boldsymbol{W}_d (\mathrm{SiLU}(\boldsymbol{a}_i) \odot \boldsymbol{b}_i) + \boldsymbol{b}_d. \end{aligned}$$

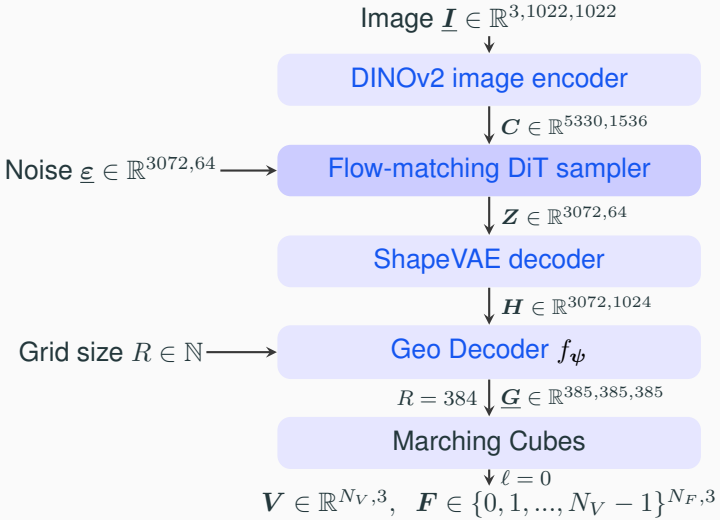
$\boldsymbol{a}_i, \boldsymbol{b}_i \in \mathbb{R}^{4096}$, $\boldsymbol{o}_i \in \mathbb{R}^{1536}$.

$$\mathrm{SiLU}(x) = \frac{x}{1 + e^{-x}}.$$



Source: `Dinov2SwiGLUFFN`.

Hunyuan3D 2.0's pipeline



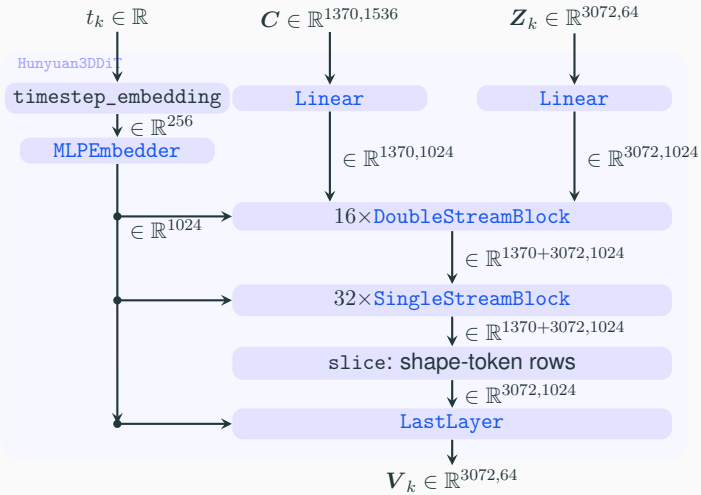
Hunyuan3DDiTFlowMatchingPipeline

For image condition $\boldsymbol{C} \in \mathbb{R}^{1370,1536}$ and $\boldsymbol{Z}_0 \in \mathbb{R}^{3072,64}$, the flow-matching sampler iterates

$$\boldsymbol{V}_k = \texttt{Hunyuan3DDiT}(\boldsymbol{Z}_k, t_k, \boldsymbol{C}) \in \mathbb{R}^{3072,64},$$
$$\boldsymbol{Z}_{k+1} = \boldsymbol{Z}_k + (t_{k+1} - t_k)\boldsymbol{V}_k, \qquad k = 0, \ldots, K-1.$$

Source: `Hunyuan3DDiTFlowMatchingPipeline`; `Hunyuan3DDiT.forward`.

Hunyuan3DDiT: complete forward path



Source: `Hunyuan3DDiT.__init__`; `Hunyuan3DDiT.forward`.

Linear: input projections in Hunyuan3DDiT

For a shape token $\boldsymbol{z}_i \in \mathbb{R}^{64}$ and a condition token $\boldsymbol{c}_j \in \mathbb{R}^{1536}$,

$$\boldsymbol{x}_i^{(0)} = \boldsymbol{W}_{\text{latent}} \boldsymbol{z}_i + \boldsymbol{b}_{\text{latent}} \in \mathbb{R}^{1024},$$
$$\boldsymbol{c}_j^{(0)} = \boldsymbol{W}_{\text{cond}} \boldsymbol{c}_j + \boldsymbol{b}_{\text{cond}} \in \mathbb{R}^{1024}.$$

$$\boldsymbol{W}_{\text{latent}} \in \mathbb{R}^{1024,64}, \qquad \boldsymbol{W}_{\text{cond}} \in \mathbb{R}^{1024,1536}, \qquad \boldsymbol{b}_{\text{latent}}, \boldsymbol{b}_{\text{cond}} \in \mathbb{R}^{1024}.$$

The output matrices contain 3072 and 1370 rows, respectively. The two `Linear` instances have distinct parameters.

Source: `Hunyuan3DDiT.__init__`; `torch.nn.Linear`.

timestep_embedding: sinusoidal time features

For $t_k \in \mathbb{R}$ and $a = 0, \ldots, 127$, define

$$f_a = \exp\left(-\frac{a \log 10000}{128}\right), \qquad p_{t_k,a} = \cos(1000\, t_k f_a), \qquad p_{t_k,a+128} = \sin(1000\, t_k f_a).$$

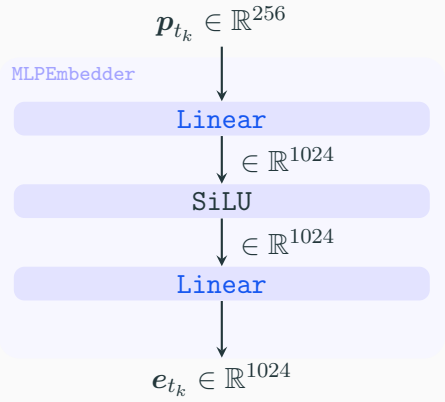
Thus $\boldsymbol{p}_{t_k} \in \mathbb{R}^{256}$. With `time_factor` $= 1000$,

$$\boldsymbol{e}_{t_k} = \texttt{MLPEmbedder}(\boldsymbol{p}_{t_k}) \in \mathbb{R}^{1024}.$$

The sinusoidal embedding has no trainable parameters; its output is computed anew from the scalar timestep supplied by the sampler.

Source: `timestep_embedding`; `Hunyuan3DDiT.forward`.

MLPEmbedder



$$\boldsymbol{u} = \boldsymbol{W}_1 \boldsymbol{p}_{t_k} + \boldsymbol{b}_1,$$
$$\boldsymbol{e}_{t_k} = \boldsymbol{W}_2 \,\mathrm{SiLU}(\boldsymbol{u}) + \boldsymbol{b}_2.$$

$$\boldsymbol{W}_1 \in \mathbb{R}^{1024,256},$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{1024,1024},$$
$$\boldsymbol{b}_1, \boldsymbol{b}_2 \in \mathbb{R}^{1024}.$$

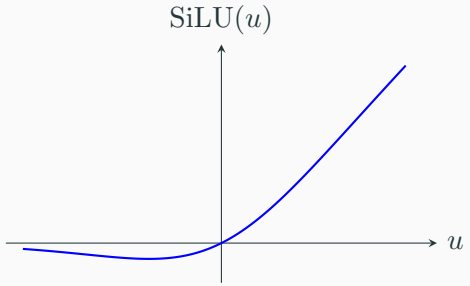
Source: `MLPEmbedder`.

SiLU

For $u \in \mathbb{R}$,

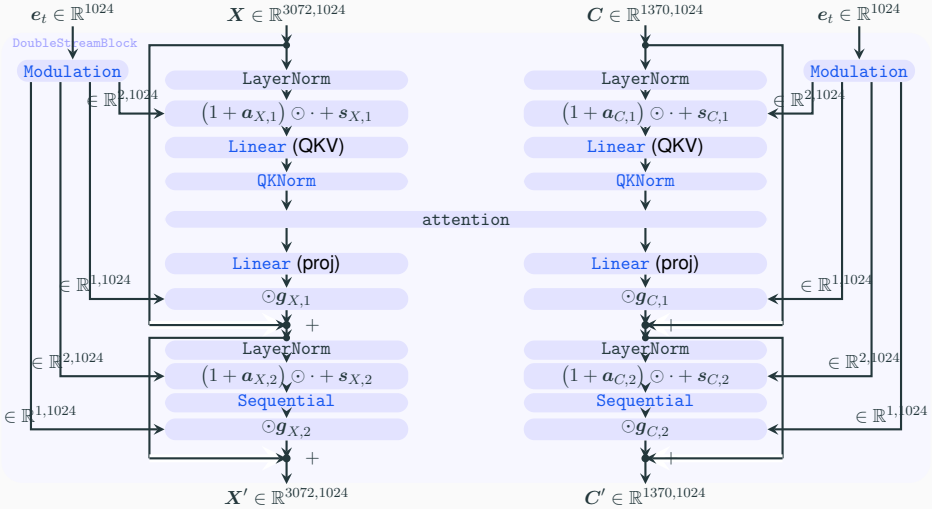
$$\mathrm{SiLU}(u) = \frac{u}{1 + \exp(-u)}.$$

The function is applied componentwise to $\boldsymbol{u} \in \mathbb{R}^{1024}$ and returns a vector in $\mathbb{R}^{1024}$. It has no trainable parameters.



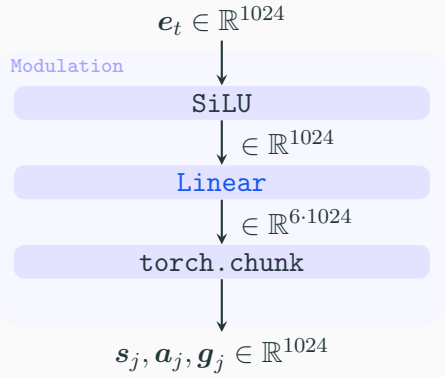
Source: `MLPEmbedder`; `SiLU`.

DoubleStreamBlock: complete forward path



Source: `DoubleStreamBlock`; v2.0 configuration.

Modulation: time-dependent shift, scale, gate



For `double=True`,

$$m = W_m \,\mathrm{SiLU}(e_t) + b_m \in \mathbb{R}^{6144}.$$

$$W_m \in \mathbb{R}^{6144,1024}, \qquad b_m \in \mathbb{R}^{6144}.$$

Six consecutive vectors yield $(s_1, a_1, g_1, s_2, a_2, g_2)$. For `double=False`, the `Linear` output has $3 \cdot 1024$ components and returns one triple.

Source: `Modulation`.

LayerNorm: without affine parameters

For $r \in \{X, C\}$, token i, and $\boldsymbol{h}_{r,i} \in \mathbb{R}^{1024}$, define

$$\mu_{r,i} = \frac{1}{1024} \sum_{a=1}^{1024} h_{r,i,a},$$

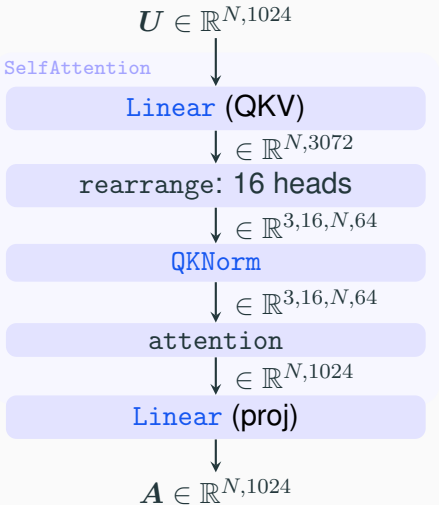
$$\nu_{r,i} = \frac{1}{1024} \sum_{a=1}^{1024} (h_{r,i,a} - \mu_{r,i})^2,$$

$$\mathrm{LayerNorm}(\boldsymbol{h}_{r,i}) = \frac{\boldsymbol{h}_{r,i} - \mu_{r,i}\mathbf{1}}{\sqrt{\nu_{r,i} + 10^{-6}}} \in \mathbb{R}^{1024}.$$

The `LayerNorm` modules in `DoubleStreamBlock` and `SingleStreamBlock` are instantiated with `elementwise_affine=False`; therefore, they do not contain learned scale or bias vectors. The separate `Modulation` determines $\boldsymbol{a}_{r,j}, \boldsymbol{s}_{r,j} \in \mathbb{R}^{1024}$ from $\boldsymbol{e}_t$.

Source: `DoubleStreamBlock.__init__`; `SingleStreamBlock`.

SelfAttention: complete forward computation



`SelfAttention.forward` performs this sequence for one input of length N.

In `DoubleStreamBlock.forward`, the two independent `SelfAttention` objects supply only their `qkv`, `norm` and `proj` submodules. Their complete `forward` methods are not called.

The `attention` function instead receives concatenated Q, K, V from both streams.

Source: `SelfAttention`, `DoubleStreamBlock.forward`

Linear: QKV projection per token

For stream $r \in \{X, C\}$, token $\boldsymbol{u}_{r,i} \in \mathbb{R}^{1024}$, and $N_X = 3072$, $N_C = 1370$,

$$\begin{pmatrix} \boldsymbol{q}_{r,i} \\ \boldsymbol{k}_{r,i} \\ \boldsymbol{v}_{r,i} \end{pmatrix} = \boldsymbol{W}^{r}_{\mathsf{QKV}} \boldsymbol{u}_{r,i} + \boldsymbol{b}^{r}_{\mathsf{QKV}} \in \mathbb{R}^{3072},$$

$$\boldsymbol{W}^{r}_{\mathsf{QKV}} \in \mathbb{R}^{3072,1024}, \quad \boldsymbol{b}^{r}_{\mathsf{QKV}} \in \mathbb{R}^{3072}, \quad \boldsymbol{q}_{r,i}, \boldsymbol{k}_{r,i}, \boldsymbol{v}_{r,i} \in \mathbb{R}^{1024}.$$

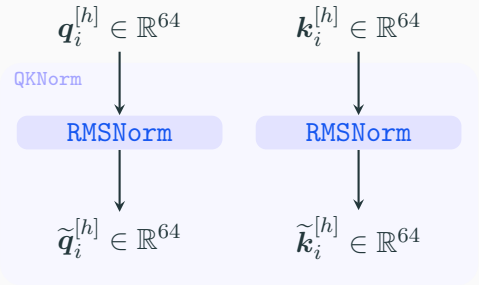
For each head $h \in \{1, \ldots, 16\}$, split the channel dimension:

$$\boldsymbol{q}^{[h]}_{r,i}, \boldsymbol{k}^{[h]}_{r,i}, \boldsymbol{v}^{[h]}_{r,i} \in \mathbb{R}^{64}, \qquad 1024 = 16 \cdot 64.$$

The two streams have separate learned matrices and biases; `qkv_bias=True` in the released v2.0 configuration.

Source: `SelfAttention.__init__`; v2.0 configuration.

QKNorm: query and key normalizers



Each RMSNorm has an independent learned $\boldsymbol{w} \in \mathbb{R}^{64}$. The value vector $\boldsymbol{v}_i^{[h]} \in \mathbb{R}^{64}$ is passed through without normalization.

The same learned normalizers are shared across token positions, not across queries and keys.

Source: QKNorm.

RMSNorm: headwise normalization

For $\boldsymbol{q} \in \mathbb{R}^{64}$,

$$\mathrm{RMSNorm}(\boldsymbol{q}) = \boldsymbol{w} \odot \frac{\boldsymbol{q}}{\sqrt{\frac{1}{64}\sum_{a=1}^{64} q_a^2 + 10^{-6}}} \in \mathbb{R}^{64}, \qquad \boldsymbol{w} \in \mathbb{R}^{64}.$$

The implementation computes the mean square in float32 and casts the normalized values to the original dtype before applying the learned scale. No additive bias is present.

The query and key branches of `QKNorm` use separate learned scale vectors.

Source: `RMSNorm`, `QKNorm`.

attention: one joint computation over both streams

For each of Q, K, V, concatenate the condition sequence (1370 tokens) before the shape sequence (3072 tokens). For head $h \in \{1, \dots, 16\}$, the resulting matrices are

$$\boldsymbol{Q}^{[h]}, \boldsymbol{K}^{[h]}, \boldsymbol{V}^{[h]} \in \mathbb{R}^{4442,64}, \qquad 4442 = 1370 + 3072.$$

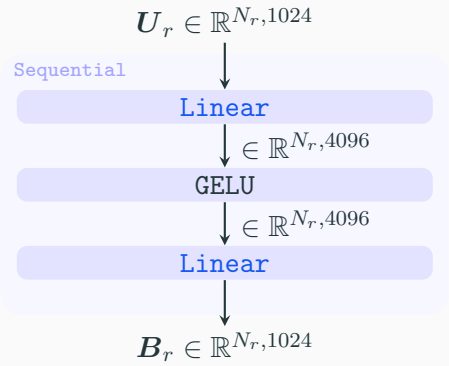
For $i, j \in \{1, \dots, 4442\}$, let $\boldsymbol{q}_i^{[h]}, \boldsymbol{k}_j^{[h]}, \boldsymbol{v}_j^{[h]} \in \mathbb{R}^{64}$ be the corresponding token-row vectors. Then

$$\alpha_{ij}^{[h]} = \frac{\exp\left((\boldsymbol{q}_i^{[h]})^\top \boldsymbol{k}_j^{[h]}/8\right)}{\sum_{a=1}^{4442} \exp\left((\boldsymbol{q}_i^{[h]})^\top \boldsymbol{k}_a^{[h]}/8\right)}, \qquad \boldsymbol{o}_i^{[h]} = \sum_{j=1}^{4442} \alpha_{ij}^{[h]} \boldsymbol{v}_j^{[h]} \in \mathbb{R}^{64}.$$

The 16 head results concatenate into $\boldsymbol{A} \in \mathbb{R}^{4442,1024}$. The first 1370 rows constitute $\boldsymbol{A}_C$ and the other 3072 rows constitute $\boldsymbol{A}_X$. The official `Hunyuan3DDiT.forward` passes `pe=None`.

Source: `attention`; `DoubleStreamBlock.forward`.

Sequential: stream-specific feed-forward network



For $\boldsymbol{u}_i \in \mathbb{R}^{1024}$,

$$\boldsymbol{w}_i = \boldsymbol{W}_1 \boldsymbol{u}_i + \boldsymbol{b}_1 \in \mathbb{R}^{4096},$$
$$\boldsymbol{b}_i = \boldsymbol{W}_2 \,\mathrm{GELU}(\boldsymbol{w}_i) + \boldsymbol{b}_2 \in \mathbb{R}^{1024}.$$

$$\boldsymbol{W}_1 \in \mathbb{R}^{4096,1024},$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{1024,4096}.$$

The X and C streams have distinct `Sequential` modules.

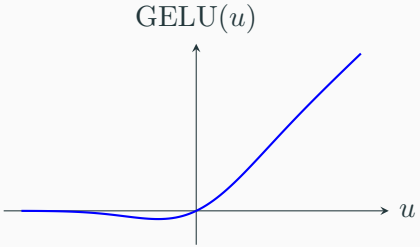
Source: `DoubleStreamBlock.__init__`.

GELU: tanh approximation

The `GELU(approximate="tanh")` class computes

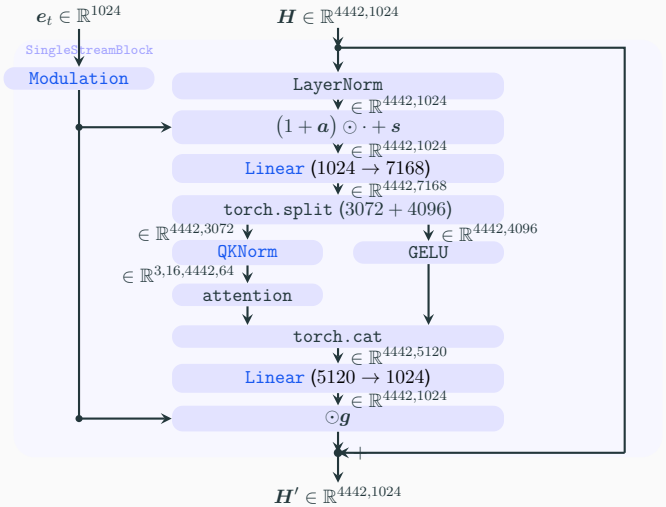
$$\mathrm{GELU}(u) = \frac{u}{2}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(u + 0.044715u^3)\right)\right), \qquad u \in \mathbb{R}.$$

The same scalar function is applied componentwise to $\boldsymbol{w}_i \in \mathbb{R}^{4096}$, producing $\mathrm{GELU}(\boldsymbol{w}_i) \in \mathbb{R}^{4096}$. The activation has no learned parameters.



Source: `GELU`.

SingleStreamBlock: complete forward path



Source: `SingleStreamBlock`.

Linear: parallel QKV and MLP-channel projections

For each modulated token $\boldsymbol{u}_i \in \mathbb{R}^{1024}$, the first learned projection returns

$$\begin{pmatrix} \boldsymbol{qkv}_i \\ \boldsymbol{m}_i \end{pmatrix} = \boldsymbol{W}_1 \boldsymbol{u}_i + \boldsymbol{b}_1 \in \mathbb{R}^{7168}, \qquad \boldsymbol{qkv}_i \in \mathbb{R}^{3072}, \quad \boldsymbol{m}_i \in \mathbb{R}^{4096}.$$

For the output of `attention`, $\boldsymbol{o}_i \in \mathbb{R}^{1024}$, the output projection and the gated residual are

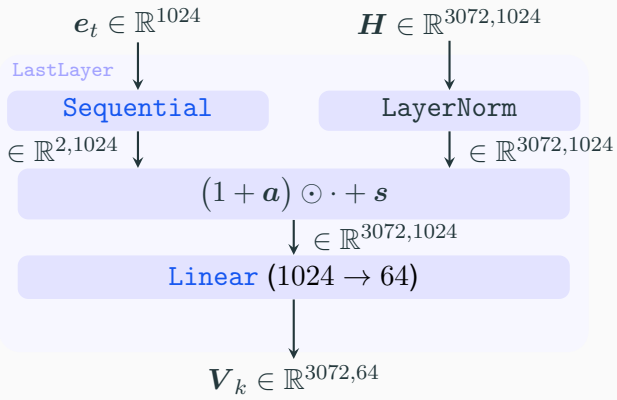
$$\boldsymbol{h}_i' = \boldsymbol{h}_i + \boldsymbol{g} \odot \left[\boldsymbol{W}_2 \begin{pmatrix} \boldsymbol{o}_i \\ \mathrm{GELU}(\boldsymbol{m}_i) \end{pmatrix} + \boldsymbol{b}_2 \right] \in \mathbb{R}^{1024}.$$

The learned parameter shapes are

$$\boldsymbol{W}_1 \in \mathbb{R}^{7168,1024}, \quad \boldsymbol{b}_1 \in \mathbb{R}^{7168},$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{1024,5120}, \quad \boldsymbol{b}_2 \in \mathbb{R}^{1024}.$$

Source: `SingleStreamBlock`.

LastLayer: complete forward path



The `Sequential` contains `SiLU` and a `Linear` mapping $1024 \to 2048$.
It returns $s, a \in \mathbb{R}^{1024}$.
With $h_i \in \mathbb{R}^{1024}$,

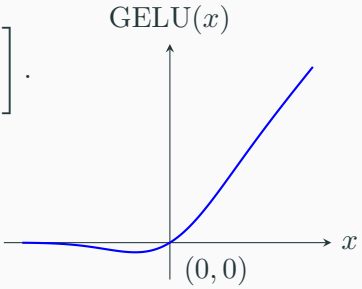
$$u_i = (\mathbf{1} + a) \odot \mathrm{LayerNorm}(h_i) + s \in \mathbb{R}^{1024},$$

$$v_i = W u_i + b \in \mathbb{R}^{64}.$$

Source: `LastLayer`; `Hunyuan3DDiT.final_layer`.

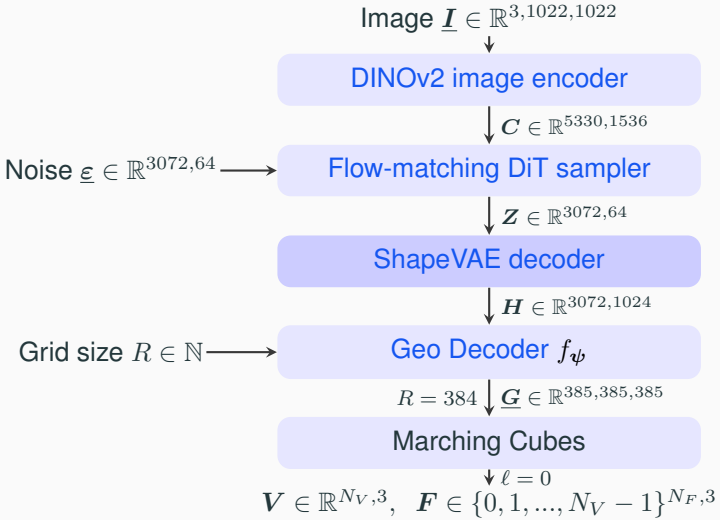
GELU: scalar nonlinearity

$$\mathrm{GELU}_{\tanh}(x) = \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right].$$

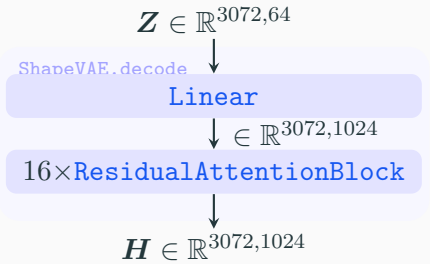


Source: `GELU.forward`.

Hunyuan3D 2.0's pipeline



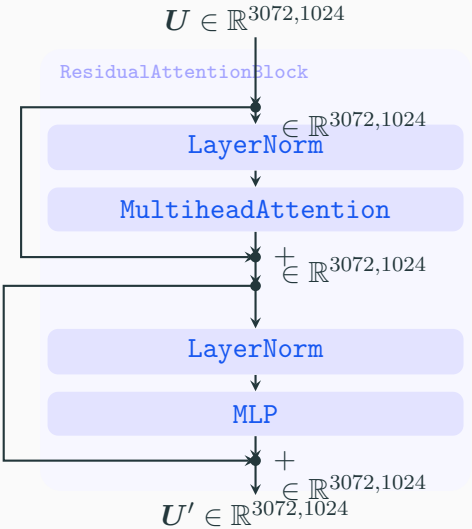
ShapeVAE.decode: projection and 16 x ResidualAttentionBlock



- No dependence on grid size R or query points $\boldsymbol{X}$.

Source: `ShapeVAE.decode`.

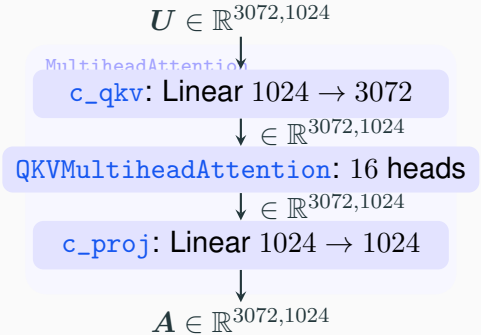
ResidualAttentionBlock: two skip additions



- $\boldsymbol{U}_1 = \boldsymbol{U} + \mathrm{Attn}(\mathrm{LN}_1(\boldsymbol{U}))$.
- $\boldsymbol{U}' = \boldsymbol{U}_1 + \mathrm{MLP}(\mathrm{LN}_2(\boldsymbol{U}_1))$.
- All skip connections require matched channel and token dimensions.

Source: `ResidualAttentionBlock.forward`.

MultiheadAttention: projections and attention



Source: `MultiheadAttention.forward`.

- QKV are split into 16 heads of 64 features each.
- $\boldsymbol{W}_{QKV} \in \mathbb{R}^{3072,1024}$; $\boldsymbol{W}_{O} \in \mathbb{R}^{1024,1024}$.
- QK layer normalization is enabled in the released v2.0 checkpoint.

QKVMultiheadAttention: vector equations

Let $L = 3072$, $h \in \{1, \ldots, 16\}$ and $d_h = 64$. For token i,

$$\boldsymbol{q}_{i,h}, \boldsymbol{k}_{i,h}, \boldsymbol{v}_{i,h} \in \mathbb{R}^{64}, \quad \boldsymbol{q}'_{i,h} = \mathrm{LN}_{Q,h}(\boldsymbol{q}_{i,h}), \quad \boldsymbol{k}'_{i,h} = \mathrm{LN}_{K,h}(\boldsymbol{k}_{i,h}).$$

$$a_{ij}^{(h)} = \frac{\exp\left(\boldsymbol{q}'_{i,h} \cdot \boldsymbol{k}'_{j,h}/8\right)}{\sum_{r=1}^{L} \exp\left(\boldsymbol{q}'_{i,h} \cdot \boldsymbol{k}'_{r,h}/8\right)}, \qquad \boldsymbol{o}_{i,h} = \sum_{j=1}^{L} a_{ij}^{(h)} \boldsymbol{v}_j^{[h]}.$$

$$\boldsymbol{o}_i = \mathrm{Concat}_{h=1}^{16}(\boldsymbol{o}_{i,h}) \in \mathbb{R}^{1024}.$$

The output is projected by `c_proj`. The layer norms have learned scale and shift vectors $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{64}$.

Source: `QKVMultiheadAttention`.

MLP and LayerNorm: component equations

For each token $\boldsymbol{x}_i \in \mathbb{R}^{1024}$, `MLP` computes

$$\boldsymbol{a}_i = \boldsymbol{W}_1 \boldsymbol{x}_i + \boldsymbol{b}_1 \in \mathbb{R}^{4096}, \qquad \boldsymbol{y}_i = \boldsymbol{W}_2 \,\mathrm{GELU}(\boldsymbol{a}_i) + \boldsymbol{b}_2 \in \mathbb{R}^{1024},$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{4096,1024}$ and $\boldsymbol{W}_2 \in \mathbb{R}^{1024,4096}$. For `LayerNorm` on $\boldsymbol{x} \in \mathbb{R}^d$,

$$\mu(\boldsymbol{x}) = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad \sigma^2(\boldsymbol{x}) = \frac{1}{d}\sum_{j=1}^{d} (x_j - \mu)^2,$$

$$\mathrm{LN}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x} - \mu(\boldsymbol{x})\mathbf{1}}{\sqrt{\sigma^2(\boldsymbol{x}) + 10^{-6}}} + \boldsymbol{\beta} \in \mathbb{R}^d.$$

In `ResidualAttentionBlock`, $d = 1024$.

Source: `MLP`.